

RAPPORT TECHNIQUE
RÉALISÉ PAR SIHAM AIT TALEB

Internet des Objets (IoT)

Système de Réfrigération Médicale Intelligente avec Supervision IoT

Architecture Modulaire et Traçabilité des Données

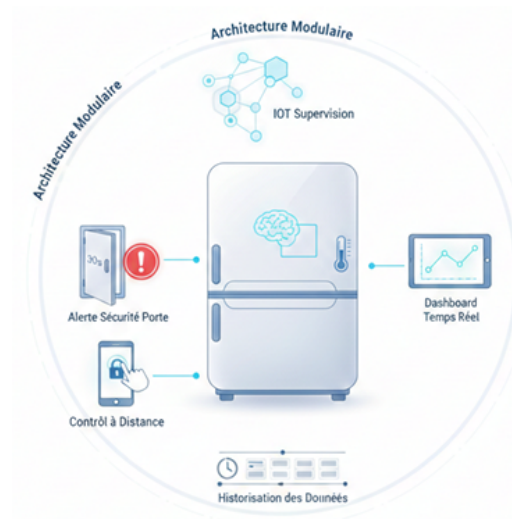


Table des matières

1	INTRODUCTION ET OBJECTIFS DU PROJET	5
1.1	Contexte	5
1.2	Objectifs du Projet	5
1.3	Technologies Utilisées	6
2	CHOIX DU MATÉRIEL ET DESCRIPTION DE LA RÉALISATION	6
2.1	Microcontrôleur ESP32	6
2.2	Capteur de Température DHT11	6
2.3	Module de Refroidissement Peltier TEC1-12706	7
2.4	Capteur d'État de Porte (Reed Switch KY-025)	7
2.5	Système d'Alerte (Buzzer + LED)	8
2.6	Servomoteur SG90 (Commande de Porte)	8
2.7	Capteur de Présence de Contenu (KY-032)	8
2.8	Position INDOR : Fingerprinting Wi-Fi - Principe	8
3	ARCHITECTURE IoT ET COMMUNICATION	9
3.1	Vue d'Ensemble de l'Architecture	9
3.2	Protocole MQTT et Broker Eclipse Mosquitto	9
3.3	Topics MQTT - Architecture et Convention	10
3.4	Configuration WiFi de l'ESP32	11
4	LOGIQUE DE RÉGULATION THERMIQUE	11
4.1	Modes Médicaux et Températures Cibles	11
4.2	Logique Conditionnelle de Commande du Peltier	11
4.3	Implémentation Logicielle	13
5	Conception matérielle et flux de fonctionnement	13
5.1	SCHÉMA DE CÂBLAGE ÉLECTRIQUE	13
5.2	Logigramme :	15
6	INTERFACE DE SUPERVISION NODE-RED	16
6.1	Présentation de Node-RED	16
6.2	Flux MQTT - Réception des Données Capteurs	16
6.3	Flux MQTT - Envoi des Commandes	17
6.4	Dashboard - Interface Utilisateur	17
6.5	Logique d'Alerte - Porte Ouverte	17

7	STOCKAGE DES DONNÉES AVEC INFLUXDB	18
7.1	Pourquoi InfluxDB?	18
7.2	Configuration	18
7.3	Flux Node-RED vers InfluxDB	19
7.4	Structure et exploitation des Données Stockées	19
8	Synthèse des Résultats	20
8.1	Validation des résultats	20
8.2	Bilan du Projet	20
8.3	Ouverture et Applications Futures	21
9	Conclusion Générale	22
10	ANNEXES	23

Table des figures

1	Architecture MQTT du projet	10
2	Circuit de commande PELTIER	13
3	Schéma de câblage complet	14
4	Logigramme	15
5	Dashboard	17
6	Dashboard + Alarme	18
7	Configuration InfluxDB	19
8	Flow Node-RED pour l'envoi des données vers InfluxDB	19
9	Base de données	20

Liste des tableaux

1	Topics MQTT de publication	10
2	Topics MQTT de souscription	10
3	Modes de conservation et paramètres thermiques	11
4	Synthèse des tests de validation	20

ABSTRACT

This technical report presents the design and implementation of an intelligent medical refrigerator system based on IoT technologies. The system integrates an ESP32 microcontroller with DHT11 temperature sensor, Peltier TEC1-12706 cooling module, and multiple security sensors (reed switch, obstacle detector). Communication is established via MQTT protocol using Eclipse Mosquitto broker, with real-time visualization through Node-RED dashboard and historical data storage in InfluxDB. The system implements conditional cooling logic based on content presence and temperature thresholds for different medical storage modes (blood, vaccines, reagents, platelets). Indoor positioning is achieved through Wi-Fi fingerprinting using RSSI values. The project demonstrates a complete IoT chain from sensing to data storage, ensuring medical cold chain compliance and remote monitoring capabilities.

Résumé

Ce rapport technique présente la conception et la réalisation d'un système de réfrigération médicale intelligent basé sur les technologies IoT. Le système intègre un microcontrôleur ESP32 avec capteur de température DHT11, module de refroidissement Peltier TEC1-12706, et plusieurs capteurs de sécurité (reed switch, détecteur d'obstacle). La communication s'établit via le protocole MQTT utilisant le broker Eclipse Mosquitto, avec visualisation temps réel sur tableau de bord Node-RED et stockage historique dans InfluxDB. Le système implémente une logique de refroidissement conditionnelle basée sur la présence du contenu et les seuils de température pour différents modes de stockage médical (sang, vaccins, réactifs, plaquettes). Le positionnement indoor est réalisé par fingerprinting Wi-Fi utilisant les valeurs RSSI. Le projet démontre une chaîne IoT complète de la capture à l'archivage des données, garantissant la conformité de la chaîne du froid médicale et les capacités de supervision à distance.

1 INTRODUCTION ET OBJECTIFS DU PROJET

1.1 Contexte

Le respect de la chaîne du froid est un enjeu critique dans le domaine médical. Les produits biologiques tels que le sang, les vaccins, les plaquettes sanguines et les réactifs nécessitent des conditions de conservation strictement contrôlées pour maintenir leur efficacité et leur sécurité. Une rupture de cette chaîne peut entraîner la dégradation des produits et compromettre la santé des patients.

Les systèmes de réfrigération traditionnels présentent plusieurs limitations :

- Absence de supervision en temps réel
- Manque de traçabilité des conditions de stockage
- Alertes de sécurité limitées ou inexistantes
- Impossibilité de contrôle à distance

1.2 Objectifs du Projet

Ce projet vise à développer un **réfrigérateur médical intelligent** intégrant les technologies IoT pour répondre aux exigences suivantes :

Objectifs fonctionnels :

- Régulation thermique adaptative selon le type de produit médical stocké
- Supervision en temps réel de la température et de l'état du système
- Système d'alerte en cas d'ouverture prolongée de la porte
- Contrôle à distance de l'ouverture/fermeture
- Détection automatique de la présence de contenu

Objectifs techniques :

- Architecture IoT complète : capteurs → communication → supervision/commande → stockage
- Communication MQTT pour l'interopérabilité
- Interface de visualisation ergonomique (Node-RED)
- Historisation des données pour traçabilité réglementaire
- Localisation indoor par fingerprinting Wi-Fi

1.3 Technologies Utilisées

Le projet s'appuie sur un ensemble cohérent de technologies open-source et standards de l'industrie :

- **Microcontrôleur** : ESP32 « NodeMCU V1.2 AI THINKER »
- **Protocole de communication** : MQTT
- **Broker MQTT** : Eclipse Mosquitto
- **Plateforme de supervision** : Node-RED
- **Base de données temporelle** : InfluxDB
- **Positionnement** : Fingerprinting Wi-Fi (RSSI)

2 CHOIX DU MATÉRIEL ET DESCRIPTION DE LA RÉALISATION

2.1 Microcontrôleur ESP32

- Connectivité WiFi intégrée (essentielle pour l'architecture IoT)
- Double cœur (permet le traitement parallèle)
- Nombreuses GPIO disponibles
- Faible consommation énergétique
- Support natif des protocoles de communication IoT

2.2 Capteur de Température DHT11

Rôle dans le système : Le DHT11 assure la mesure de la température interne du réfrigérateur, donnée critique pour la régulation thermique.

Caractéristiques techniques :

- Plage de température : 0°C à 50°C
- Précision : $\pm 2^\circ\text{C}$
- Plage d'humidité : 20% à 80% RH
- Interface : Digital (protocole propriétaire 1-wire)
- Temps de réponse : $< 6\text{s}$
- Alimentation : 3.3V - 5V

2.3 Module de Refroidissement Peltier TEC1-12706

Principe de fonctionnement : Le module Peltier utilise l'effet thermoélectrique (effet Peltier) pour créer une différence de température entre ses deux faces lorsqu'il est alimenté en courant continu. Une face se refroidit (intérieur du réfrigérateur) tandis que l'autre se réchauffe (dissipation par heat sink).

Caractéristiques techniques :

- Modèle : TEC1-12706
- Tension nominale : 12V DC
- Courant maximal : 6A
- Puissance maximale : 72W
- Différentiel de température : jusqu'à 60°C

Note : L'ESP32 délivre un signal logique de 3.3V avec un courant limité (quelques mA), insuffisant pour commander directement le MOSFET de puissance pilotant le Peltier (qui nécessite typiquement 10V+ en gate pour une conduction optimale).

Solution retenue : Étage d'adaptation avec transistor

Composants :

- **MOSFET IRFZ44N** : canal N, 55V, 49A, $R_{ds(on)} = 17.5m\Omega$
- **Transistor 2N2222A** : NPN, usage général, gain typique = 100-300
- **Résistance 10k (R1)** : pull-down entre Gate et Source du MOSFET
- **Résistance de base (R2)** : entre GPIO ESP32 et base du 2N2222A

2.4 Capteur d'État de Porte (Reed Switch KY-025)

Principe de fonctionnement : Le reed switch est un interrupteur magnétique constitué de deux lames ferromagnétiques scellées dans un tube de verre. En présence d'un champ magnétique (aimant), les lames se touchent et ferment le circuit.

Application dans le projet :

- Reed switch fixé sur le cadre du réfrigérateur
- Aimant fixé sur la porte
- Porte fermée → aimant proche du reed switch → circuit fermé → lecture LOW
- Porte ouverte → aimant éloigné → circuit ouvert → lecture HIGH (via pull-up interne)

2.5 Système d'Alerte (Buzzer + LED)

Composants :

- **Buzzer passif** : permet la génération de mélodies (fréquences variables)
- **LED rouge** : signalisation visuelle

Logique d'alerte : L'alarme se déclenche lorsque la porte reste ouverte plus de 30 secondes consécutives. Une mélodie alternée (2000Hz / 800Hz) est générée pour attirer l'attention, accompagnée du clignotement de la LED rouge.

2.6 Servomoteur SG90 (Commande de Porte)

Rôle : Permet l'ouverture et la fermeture motorisée de la porte du réfrigérateur via commande à distance depuis Node-RED.

Caractéristiques :

- Couple : 1.8 kg·cm (4.8V)
- Angle de rotation : 180°
- Temps de réponse : 0.1s / 60°
- Commande : signal PWM (50Hz)

2.7 Capteur de Présence de Contenu (KY-032)

Principe : Capteur infrarouge d'obstacle détectant la présence d'un objet à courte distance (2-30cm).

Rôle dans le système : Détecte si le réfrigérateur contient des produits médicaux. Cette information est critique pour la logique de commande du Peltier, pour des raisons d'économisation d'énergie.

2.8 Position INDOR : Fingerprinting Wi-Fi - Principe

Le fingerprinting Wi-Fi est une technique de localisation indoor basée sur la **cartographie des signaux Wi-Fi** (RSSI - Received Signal Strength Indicator).

Principe en deux phases :

Phase 1 : Calibration (Offline)

1. Division du bâtiment en zones/points de référence
2. Mesure des RSSI de tous les points d'accès Wi-Fi visibles depuis chaque point
3. Constitution d'une base de données : (Point, [RSSI_AP1, RSSI_AP2, ..., RSSI_APn])

Phase 2 : Localisation (Online)

1. Mesure des RSSI actuels depuis le device (ESP32)
2. Comparaison avec la base de données (algorithme k-NN, bayésien, etc.)
3. Estimation de la position : point de référence le plus proche

Avantages :

- Fonctionne en intérieur
- Utilise l'infrastructure Wi-Fi existante (pas de matériel supplémentaire)
- Faible consommation (scan Wi-Fi passif)

3 ARCHITECTURE IoT ET COMMUNICATION

3.1 Vue d'Ensemble de l'Architecture

Le système repose sur une architecture IoT en couches respectant le modèle standard :

- **Couche Perception (Capteurs/Actionneurs)** → ESP32 + DHT11 + Reed Switch + KY-032 + Servo + Peltier
- **Couche Réseau (Communication)** → WiFi + MQTT (Mosquitto)
- **Couche Application (Traitement)** → Node-RED
- **Couche Stockage (Persistance)** → InfluxDB

Cette architecture garantit la modularité, l'interopérabilité et l'évolutivité du système.

3.2 Protocole MQTT et Broker Eclipse Mosquitto

Pourquoi MQTT ? MQTT (Message Queuing Telemetry Transport) est le protocole standard pour l'IoT, particulièrement adapté aux réseaux à faible bande passante et aux systèmes embarqués :

- Léger et efficace (overhead minimal)
- Modèle publish/subscribe (découplage producteur/consommateur)
- QoS (Quality of Service) configurable
- Support natif dans Node-RED

Architecture MQTT du projet :



FIGURE 1 – Architecture MQTT du projet

Broker Eclipse Mosquitto :

- Version utilisée : dernière version stable
- Configuration : locale (127.0.0.1)
- Port : 1883 (port MQTT standard non sécurisé)
- Mode : bridge désactivé (système local)
- Authentification : désactivée (projet académique)

3.3 Topics MQTT - Architecture et Convention

Principe de nommage : La structure des topics suit une hiérarchie logique : domaine/sous-système/type

Topics de publication (ESP32 → Node-RED) :

Topic	Type	Description
frigo/temp/current	float	Température actuelle mesurée par DHT11 (°C)
frigo/door/status	string	État de la porte : "OPEN" ou "CLOSED"
frigo/obstacle	string	Présence de contenu : "PLEIN" ou "VIDE"
frigo/peltier/state	string	État du Peltier : "ON" ou "OFF"
frigo/gps/lat	float	Latitude (fingerprinting Wi-Fi)
frigo/gps/lng	float	Longitude (fingerprinting Wi-Fi)

TABLE 1 – Topics MQTT de publication

Topics de souscription (ESP32 ← Node-RED) :

Topic	Type	Description
frigo/mode/set	string	Commande de changement de mode médical
frigo/servo/cmd	string	Commande d'ouverture/fermeture de la porte

TABLE 2 – Topics MQTT de souscription

3.4 Configuration WiFi de l'ESP32

Logique de connexion :

1. Au démarrage, l'ESP32 tente de se connecter au réseau WiFi
2. Attente active jusqu'à connexion réussie
3. Une fois connecté, connexion au broker MQTT
4. En cas de déconnexion WiFi ou MQTT, reconnexion automatique

4 LOGIQUE DE RÉGULATION THERMIQUE

4.1 Modes Médicaux et Températures Cibles

Le système supporte quatre modes de conservation médicale, chacun avec sa température cible et sa bande d'hystérésis optimisées :

Mode	Produit	Temp. Cible	Hystérésis	Plage Effective
SANG	Poches de sang	+4°C	±2°C	2°C - 6°C
VACCINS	Vaccins standards	+5°C	±3°C	2°C - 8°C
REACTIFS	Réactifs de laboratoire	+10°C	±2°C	8°C - 12°C
PLAQUETTES	Plaquettes sanguines	+23°C	±2°C	21°C - 25°C

TABLE 3 – Modes de conservation et paramètres thermiques

4.2 Logique Conditionnelle de Commande du Peltier

Principe fondamental : La commande du module Peltier repose sur une logique à double condition hiérarchique pour optimiser la consommation énergétique et la durée de vie du module.

Condition 1 (prioritaire) : Présence de contenu

Le capteur KY-032 détecte si le réfrigérateur contient des produits médicaux.

Listing 1 – Logique conditionnelle - Présence de contenu

```

1 SI frigo VIDE ALORS
2   Peltier = OFF
3   (quel que soit le mode s lectionn )
4 FIN SI

```

Justification :

- Éviter le gaspillage énergétique (refroidissement d'un espace vide)
- Préserver le module Peltier (réduction des cycles thermiques inutiles)
- Logique de sécurité opérationnelle

Condition 2 (si frigo plein) : Régulation thermique

Si et seulement si le réfrigérateur contient des produits, la régulation thermique est activée.

Listing 2 – Logique conditionnelle - Régulation thermique

```

1 SI frigo PLEIN ALORS
2   Lire temp rature DHT11
3   Calculer seuil_haut = temp rature_cible + hyst r sis
4   Calculer seuil_bas = temp rature_cible - hyst r sis
5
6   SI temp rature > seuil_haut ALORS
7     Peltier = ON (refroidissement actif)
8   SINON SI temp rature < seuil_bas ALORS
9     Peltier = OFF (temp rature suffisamment basse)
10  SINON
11    Maintenir tat actuel ( viter oscillations)
12  FIN SI
13 FIN SI

```

Exemple concret (mode SANG) :

- Température cible : 4°C
- Hystérésis : $\pm 2^\circ\text{C}$
- Seuil haut : 6°C
- Seuil bas : 2°C

Scénario :

1. Température = 7°C → Peltier ON (refroidissement)
2. Température descend progressivement
3. Température = 3°C → Peltier reste ON (hystérésis)
4. Température = 1.8°C → Peltier OFF (seuil bas atteint)
5. Température remonte naturellement
6. Température = 6.2°C → Peltier ON (seuil haut dépassé)

Avantages de l'hystérésis :

- Évite les cycles ON/OFF trop fréquents (usure prématurée)

- Réduit les oscillations thermiques
- Améliore l'efficacité énergétique

4.3 Implémentation Logicielle

L'implémentation logicielle sur l'ESP32 suit le pseudo-code suivant : Voir annexe [10](#)

5 Conception matérielle et flux de fonctionnement

5.1 SCHÉMA DE CÂBLAGE ÉLECTRIQUE

Le schéma de câblage électrique complet est représenté à ces Figures [2](#) et [3](#).

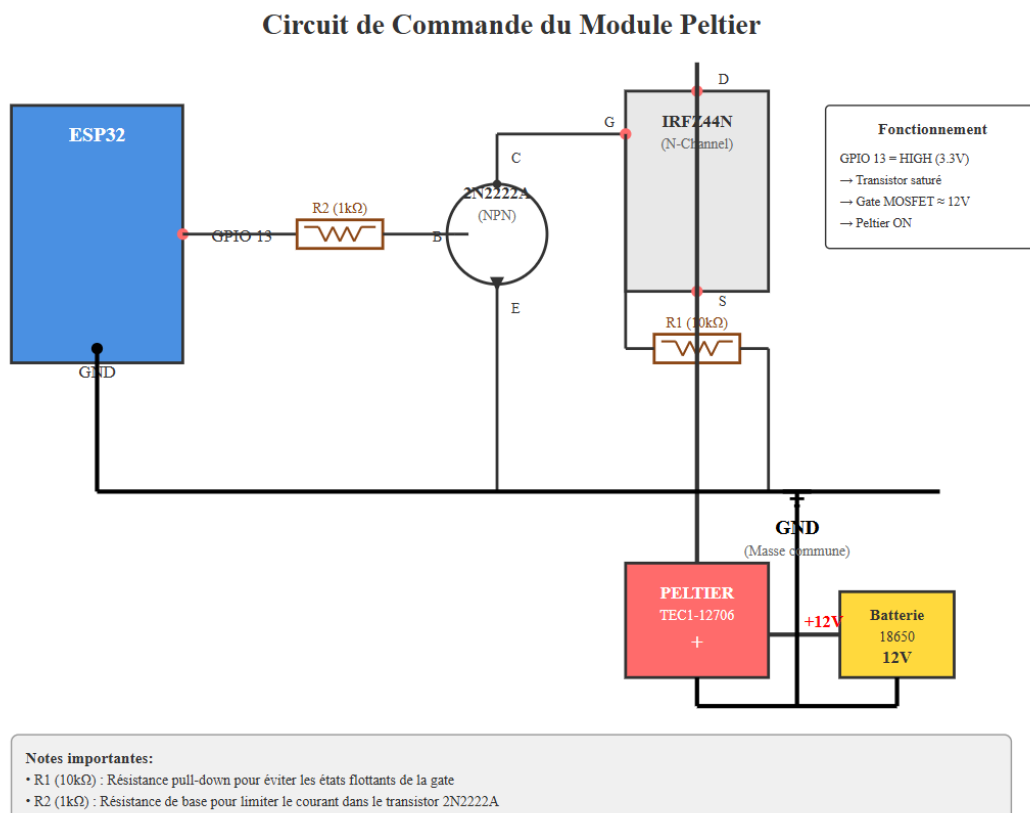


FIGURE 2 – Circuit de commande PELTIER

5.2 Logigramme :

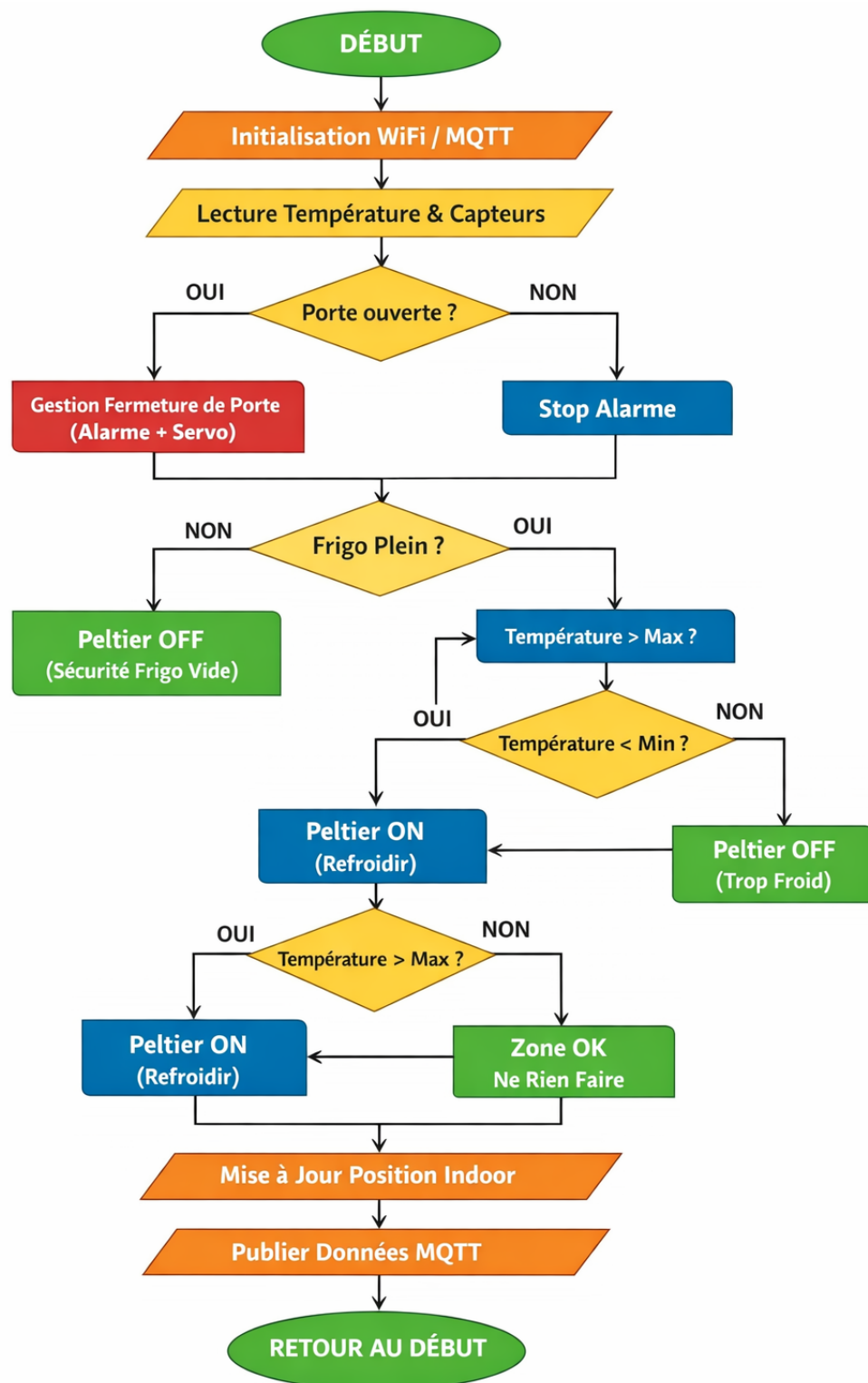


FIGURE 4 – Logigramme

6 INTERFACE DE SUPERVISION NODE-RED

6.1 Présentation de Node-RED

Node-RED est une plateforme de développement low-code orientée flux (flow-based programming), particulièrement adaptée à l'IoT :

- Interface graphique intuitive (glisser-déposer de nœuds)
- Bibliothèque riche de nœuds préconfigurés
- Intégration native MQTT, InfluxDB, HTTP, etc.
- Dashboard intégré pour visualisation temps réel
- Déploiement instantané (pas de compilation)

Point critique : Tous les nœuds MQTT du flow doivent référencer cette même configuration de broker pour assurer la cohérence de la communication.

6.2 Flux MQTT - Réception des Données Capteurs

Nœud MQTT In - Température :

- Topic : frigo/temp/current
- QoS : 2
- Output : string

Traitement de la température :

1. Réception du payload (string "4.7")
2. Conversion en nombre via nœud Function :

```
1 msg.payload = Number(msg.payload);  
2 return msg;
```

3. Distribution vers :

- Jauge de température (gauge_temp)
- Graphique historique (chart_temp)
- Stockage InfluxDB (via fonction de formatage)

Nœuds MQTT In supplémentaires :

- frigo/door/status → affichage état porte + logique d'alerte
- frigo/obstacle → affichage contenu frigo
- frigo/peltier/state → affichage état du système de refroidissement
- frigo/location → localisation indoor

6.3 Flux MQTT - Envoi des Commandes

Nœud Dropdown - Sélection du Mode : → MQTT Out (frigo/mode/set)

Nœuds Button - Commande Servo :

— Flux : Button OUVRIIR → MQTT Out (frigo/servo/cmd, payload="OPEN")

— Flux : Button FERMER → MQTT Out (frigo/servo/cmd, payload="CLOSE")

Point technique essentiel : Les nœuds MQTT Out doivent avoir leur topic configuré **avant** le nœud (dans les propriétés ou via `msg.topic`). Le payload est transmis tel quel depuis le bouton ou le dropdown.

6.4 Dashboard - Interface Utilisateur

Organisation en groupes :

Groupe 1 : System control

Groupe 2 : Statut température

Groupe 3 : Live monitoring

Groupe 4 : Positionnement Indoor

N.B : Code JSON Node-RED en annexe

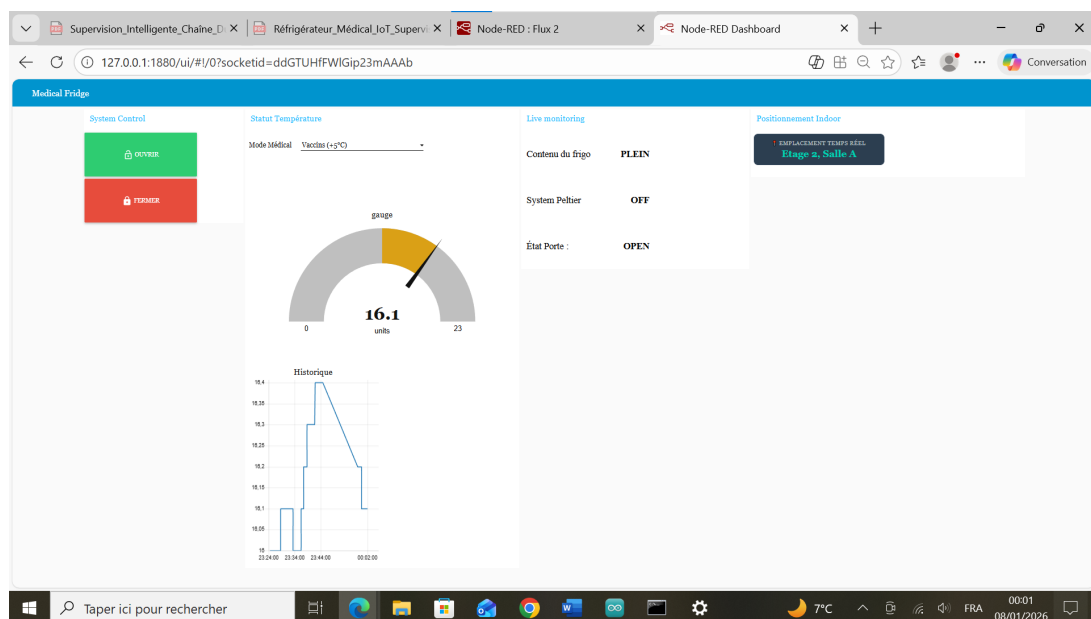


FIGURE 5 – Dashboard

6.5 Logique d'Alerte - Porte Ouverte

Nœud Trigger (door_logic) : Ce nœud implémente la temporisation de 30 secondes :

— Topic : frigo/door/status

Fonctionnement :

1. Réception "OPEN" → démarrage timer 30s
2. Si "CLOSED" reçu avant 30s → reset, pas d'alerte
3. Si timer atteint 30s → Nœud Toast (alerte visuelle sur le dashboard)

Affiche une boîte de dialogue modale pendant 10 secondes pour alerter l'utilisateur.

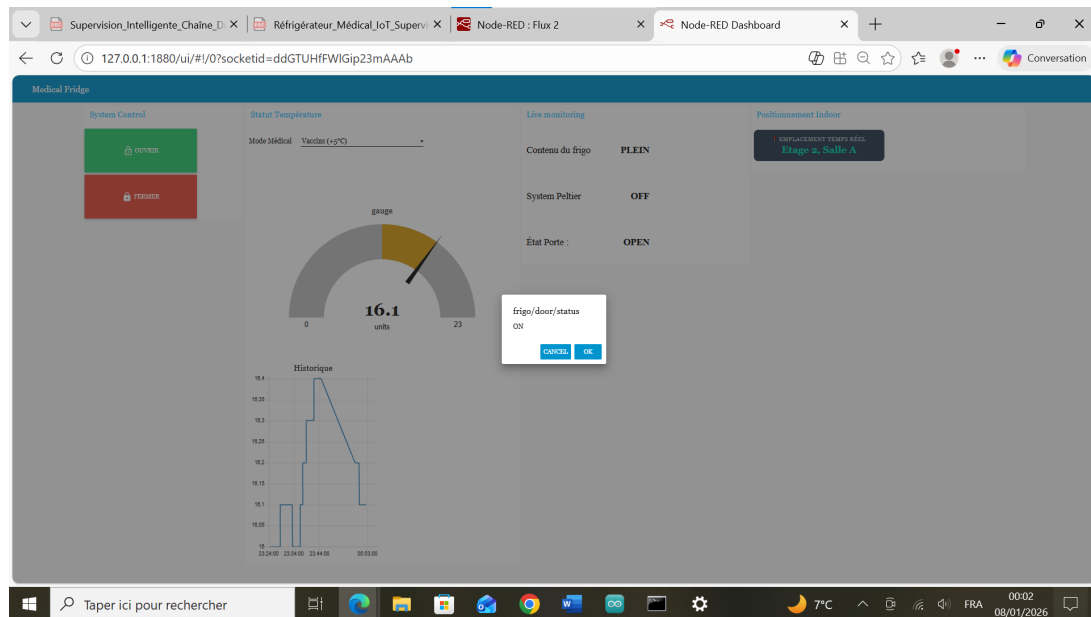


FIGURE 6 – Dashboard + Alarme

7 STOCKAGE DES DONNÉES AVEC INFLUXDB

7.1 Pourquoi InfluxDB ?

InfluxDB est une base de données optimisée pour les séries temporelles (time-series database), particulièrement adaptée aux données IoT :

- **Performances élevées** en écriture et lecture de données horodatées
- **Compression efficace** des données temporelles
- **Requêtes spécialisées** (agrégations, downsampling, retention policies)
- **Intégration native** avec Node-RED
- **Visualisation** possible avec Grafana (non implémenté dans ce projet)

7.2 Configuration

Configuration InfluxDB 2.0 :

- **URL** : `http://localhost:8086`
- **Organisation** : MedicalLab
- **Bucket** : `Fridge_Data`
- **Token** : généré lors de l'initialisation (à copier dans Node-RED)

```

-----
Your flow credentials file is encrypted using a system-generated key.

If the system-generated key is lost for any reason, your credentials
file will not be recoverable, you will have to delete it and re-enter
your credentials.

You should set your own key using the 'credentialSecret' option in
your settings file. Node-RED will then re-encrypt your credentials
file using your chosen key the next time you deploy a change.
-----

23 Dec 19:07:51 - [info] Starting flows
23 Dec 19:07:51 - [info] Started flows
23 Dec 19:07:51 - [info] [mqtt-broker:localhost] Connected to broker: mqtt://localhost:1883
23 Dec 19:08:59 - [info] Installing module: node-red-contrib-influxdb, version: 0.7.0
23 Dec 19:09:04 - [info] Installed module: node-red-contrib-influxdb
23 Dec 19:09:04 - [info] Added node types:
23 Dec 19:09:04 - [info]   - node-red-contrib-influxdb:influxdb
23 Dec 19:09:04 - [info]   - node-red-contrib-influxdb:influxdb out
23 Dec 19:09:04 - [info]   - node-red-contrib-influxdb:influxdb batch
23 Dec 19:09:04 - [info]   - node-red-contrib-influxdb:influxdb in

```

FIGURE 7 – Configuration InfluxDB

7.3 Flux Node-RED vers InfluxDB

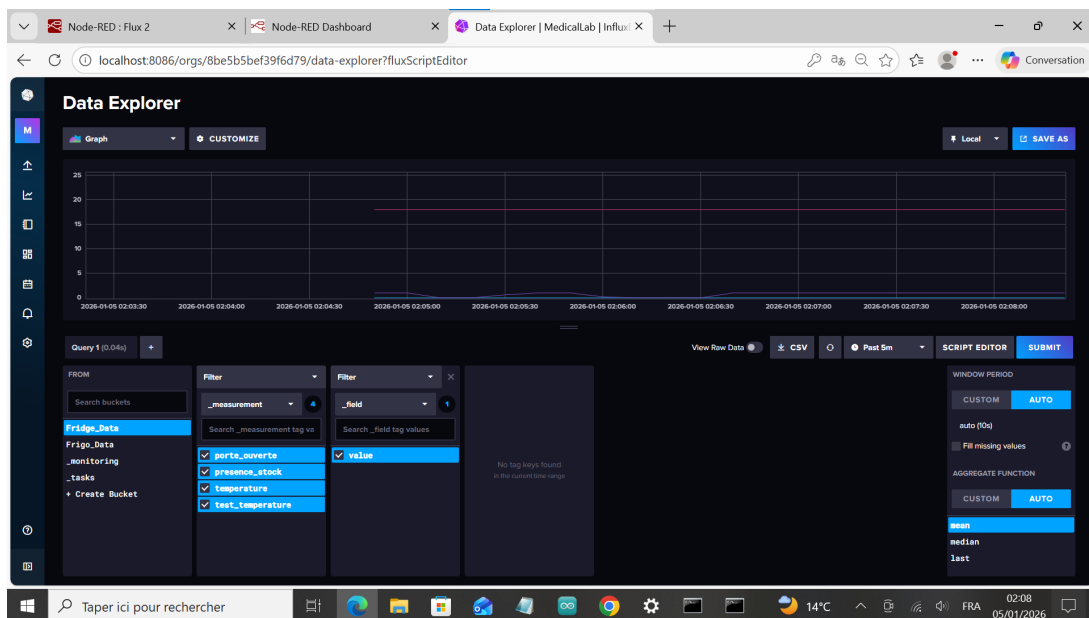


FIGURE 8 – Flow Node-RED pour l'envoi des données vers InfluxDB

7.4 Structure et exploitation des Données Stockées

Requêtes possibles (via CLI InfluxDB ou API) :

Ces données permettent la traçabilité réglementaire exigée pour le stockage de produits médicaux.

The screenshot shows an Excel spreadsheet with the following data table:

Table	start	stop	time	field	etat	porte	presence	st	systeme	pel	temperature
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:27Z	value	0	0	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:29Z	value	0	0	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:31Z	value	1	0	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:33Z	value	1	0	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:35Z	value	1	0	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:37Z	value	1	0	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:40Z	value	1	0	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:41Z	value	1	1	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:43Z	value	1	1	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:46Z	value	1	1	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:47Z	value	1	1	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:50Z	value	1	1	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:51Z	value	1	1	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:53Z	value	1	1	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:55Z	value	0	1	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:58Z	value	0	1	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:31:59Z	value	0	1	0	0	18.3		
0	2026-01-05T02:31:24.6849668Z	2026-01-05T02:32:24.6849668Z	2026-01-05T02:32:01Z	value	0	1	0	0	18.3		

FIGURE 9 – Base de données

Remarque : Pour faciliter l’exploitation des données dans des outils externes (Excel, Node-RED, etc.), la requête Flux peut être complétée par l’instruction :

```
1 |> pivot(rowKey=["_time"], columnKey: ["_measurement"], valueColumn: "_value")
```

8 Synthèse des Résultats

8.1 Validation des résultats

Composant	Statut	Commentaire
Régulation thermique	Validé	Stabilité et précision conformes
Communication MQTT	Validé	Fiabilité et latence excellentes
Interface Node-RED	Validé	Ergonomie et réactivité optimales
Stockage InfluxDB	Validé	Performance et fiabilité validées
Positionnement indoor	Validé	Précision acceptable pour l’usage
Système d’alerte	Validé	Détection et notification fonctionnelles

TABLE 4 – Synthèse des tests de validation

8.2 Bilan du Projet

Ce projet de réfrigérateur médical intelligent a permis de démontrer l’application concrète des technologies IoT dans un contexte médical exigeant. L’intégration de l’ESP32,

du protocole MQTT, de Node-RED et d’InfluxDB constitue une architecture complète et cohérente, respectant les standards de l’industrie.

Objectifs atteints :

- Régulation thermique adaptative selon le mode médical sélectionné
- Supervision en temps réel via dashboard ergonomique
- Système d’alerte de sécurité (porte ouverte > 30s)
- Contrôle à distance (ouverture/fermeture motorisée)
- Historisation des données pour traçabilité réglementaire
- Architecture modulaire et évolutive

Points forts du système :

- **Logique conditionnelle intelligente** (Peltier OFF si frigo vide)
- **Communication MQTT fiable** avec QoS 2 garantissant la livraison des messages critiques
- **Interface utilisateur intuitive** facilitant l’exploitation par du personnel non-technique
- **Traçabilité complète** des conditions de stockage (exigence réglementaire)
- **Approche pragmatique** du positionnement indoor (fingerprinting Wi-Fi)

8.3 Ouverture et Applications Futures

Ce projet ouvre la voie à plusieurs extensions possibles :

Extension 1 : Frigo multi-compartiments

- Plusieurs zones thermiques indépendantes
- Un ESP32 par zone ou multiplexage des capteurs
- Dashboard unifié pour supervision globale

Extension 2 : Système de gestion de stock

- Identification RFID des produits médicaux
- Suivi des dates de péremption
- Alertes multi-canaux :SMS (via Twilio), email, notifications push

Extension 3 : Apprentissage automatique

- Prédiction des besoins de refroidissement basée sur historique
- Détection d’anomalies (capteur défaillant, porte mal fermée)
- Optimisation énergétique par apprentissage des patterns d’utilisation

Extension 4 : Intégration avec systèmes hospitaliers

- API REST pour connexion avec logiciels de gestion hospitalière (HIS)
- Synchronisation avec systèmes de traçabilité pharmaceutique
- Intégration avec systèmes d’alerte centralisés

9 Conclusion Générale

Ce projet de réfrigérateur médical intelligent constitue une démonstration réussie de l’application des technologies IoT à un cas d’usage critique du domaine de la santé. L’architecture complète, allant de la couche capteurs jusqu’au stockage des données, en passant par une communication MQTT robuste et une interface de supervision ergonomique, illustre parfaitement les concepts fondamentaux de l’Internet des Objets.

La logique de régulation thermique conditionnelle, prenant en compte à la fois la présence de contenu et les seuils de température, démontre une approche réfléchie de l’optimisation énergétique et opérationnelle. Le système d’alerte de sécurité et la traçabilité complète des données répondent aux exigences de base d’un dispositif médical.

Bien que réalisé dans un cadre académique, ce système pose les fondations d’une solution potentiellement déployable en environnement réel moyennant les adaptations réglementaires et sécuritaires nécessaires. Les perspectives d’amélioration identifiées (cloud, précision accrue, alertes avancées, Grafana) offrent des axes de développement clairs pour transformer ce prototype fonctionnel en produit mature.

Ce projet illustre comment les technologies open-source, accessibles et peu coûteuses (ESP32, Node-RED, MQTT, InfluxDB) permettent aujourd’hui de développer des solutions IoT professionnelles, capables de répondre à des enjeux critiques tels que la préservation de la chaîne du froid médicale.

10 ANNEXES

Les annexes suivantes sont disponibles en fichiers séparés :

Annexe A : Code source complet de l'ESP32

Listing 3 – Code source principal de l'ESP32 - Version compacte

```

1  #include <WiFi.h>           // Wi-Fi
2  #include <PubSubClient.h>   // MQTT
3  #include <DHT.h>            // Capteur temp rature
4  #include <ESP32Servo.h>     // Servomoteur
5
6  // Configuration
7  const char* ssid = "DESKTOP-3JK5I2G 6286";
8  const char* password = "3o329G$0";
9  const char* mqtt_server = "192.168.137.1";
10 const char* SSID_SALLE_A = "DESKTOP-3JK5I2G 6286";
11 const char* SSID_SALLE_C = "saja1";
12
13 // Pins
14 #define DHTPIN 4             // DHT11
15 #define DHTTYPE DHT11
16 #define PELTIER_PIN 13      // Peltier
17 #define REED_SWITCH 14      // Porte
18 #define BUZZER_PIN 33       // Buzzer
19 #define LED_ALARM_PIN 32    // LED
20 #define SERVO_PIN 15        // Servo
21 #define OBSTACLE_PIN 27     // Pr sence
22
23 // Objets
24 DHT dht(DHTPIN, DHTTYPE);
25 Servo myServo;
26 WiFiClient espClient;
27 PubSubClient client(espClient);
28
29 // Variables
30 float currentTemp = NAN;
31 float targetTemp = 22.0;
32 float currentHysteresis = 3.0;
33 unsigned long lastMsg = 0;
34 String currentLocation = "Inconnue";
35 unsigned long doorOpenTime = 0;
36 bool doorTimerRunning = false;
37 bool obstacleDetected = false;
38
39 // Fonctions alarme
40 void stopAlarm() { digitalWrite(BUZZER_PIN, LOW); digitalWrite(LED_ALARM_PIN, LOW); }
41 void playPreAlarm() { static unsigned long lastBip=0; static bool state=false;
42   if (millis()-lastBip>1000){lastBip=millis(); state=!state;
43     digitalWrite(BUZZER_PIN,state); digitalWrite(LED_ALARM_PIN,state);}}
44 void playCriticalAlarm() { static unsigned long lastToggle=0; static bool state=false;
45   if (millis()-lastToggle>150){lastToggle=millis(); state=!state;
46     digitalWrite(BUZZER_PIN,state); digitalWrite(LED_ALARM_PIN,state);}}
47
48 // Positionnement WiFi
49 void updateIndoorLocation() {
50   int n=WiFi.scanNetworks(); long rssiA=-100, rssiC=-100;
51   for(int i=0;i<n;i++){ if(WiFi.SSID(i)==SSID_SALLE_A) rssiA=WiFi.RSSI(i);
52     if(WiFi.SSID(i)==SSID_SALLE_C) rssiC=WiFi.RSSI(i); }
53   if(rssiA==-100&&rssiC==-100) currentLocation="Inconnue";
54   else if(rssiA>rssiC) currentLocation="Etag 2, Salle A";
55   else currentLocation="Etag 1, Salle C";
56   WiFi.scanDelete(); }
57
58 // S curit porte
59 void handleDoorSecurity(bool doorOpen) {
60   if(doorOpen){ if(!doorTimerRunning){doorOpenTime=millis(); doorTimerRunning=true;}
61     unsigned long duration=millis()-doorOpenTime;
62     if(duration>30000){playCriticalAlarm(); myServo.write(0);}
63     else if(duration>20000){playPreAlarm();}
64   } else { doorTimerRunning=false; stopAlarm(); }}
65
66 // Callback MQTT

```



```

67 void callback(char* topic, byte* payload, unsigned int length){
68   String msg; for(unsigned int i=0;i<length;i++) msg+=(char)payload[i];
69   if(String(topic)=="frigo/mode/set"){
70     if(msg=="SANG"){targetTemp=4.0; currentHysteresis=2.0;}
71     else if(msg=="PLAQUETTES"){targetTemp=22.0; currentHysteresis=2.0;}
72     else if(msg=="VACCINS"){targetTemp=5.0; currentHysteresis=3.0;}
73     else if(msg=="REACTIFS ENZYMATIQUES"){targetTemp=10.0; currentHysteresis=2.0;}
74   }
75   if(String(topic)=="frigo/servo/cmd"){
76     if(msg=="OPEN") myServo.write(180);
77     if(msg=="CLOSE") myServo.write(0);}
78
79 void setup_wifi(){ Serial.print("Connexion WiFi");
80   WiFi.begin(ssid, password);
81   while(WiFi.status()!=WL_CONNECTED){ delay(500); Serial.print("."); }
82   Serial.println("\nWiFi connect ");}
83
84 void reconnect(){ while(!client.connected()){
85   if(client.connect("ESP32_Medical_Frigo")){
86     client.subscribe("frigo/mode/set"); client.subscribe("frigo/servo/cmd");
87   } else { delay(3000); }}
88
89 // Setup
90 void setup(){ Serial.begin(115200);
91   pinMode(PELTIER_PIN, OUTPUT); pinMode(BUZZER_PIN, OUTPUT);
92   pinMode(LED_ALARM_PIN, OUTPUT); pinMode(REED_SWITCH, INPUT_PULLUP);
93   pinMode(OBSTACLE_PIN, INPUT); digitalWrite(PELTIER_PIN, LOW);
94   stopAlarm(); myServo.attach(SERVO_PIN); myServo.write(0);
95   dht.begin(); setup_wifi();
96   client.setServer(mqtt_server, 1883); client.setCallback(callback);}
97
98 // Loop
99 void loop(){ if(!client.connected()) reconnect(); client.loop();
100   obstacleDetected=(digitalRead(OBSTACLE_PIN)==LOW);
101   bool doorOpen=(digitalRead(REED_SWITCH)==LOW);
102   currentTemp=dht.readTemperature(); handleDoorSecurity(doorOpen);
103
104   float tempMax=targetTemp+currentHysteresis;
105   float tempMin=targetTemp-currentHysteresis;
106
107   if(obstacleDetected && !isnan(currentTemp)){
108     if(currentTemp>tempMax) digitalWrite(PELTIER_PIN, LOW);
109     else if(currentTemp<tempMin) digitalWrite(PELTIER_PIN, HIGH);
110   } else { digitalWrite(PELTIER_PIN, HIGH); }
111
112   if(millis()-lastMsg>2000){ lastMsg=millis(); updateIndoorLocation();
113     Serial.printf("Temp: %.2f | Porte: %s\n", currentTemp, doorOpen?"OUVERTE":"FERMEE");
114     client.publish("frigo/temp/current",String(currentTemp).c_str());
115     client.publish("frigo/door/status",doorOpen?"OPEN":"CLOSED");
116     client.publish("frigo/obstacle",obstacleDetected?"PLEIN":"VIDE");
117     client.publish("frigo/peltier/state",digitalRead(PELTIER_PIN)?"OFF":"ON");
118     client.publish("frigo/location",currentLocation.c_str());}

```

Annexe B : Configuration complète des flows Node-RED

Listing 4 – Flux Node-RED - Version compacte

```

1 [{"id":"tab1","type":"tab","label":"Medical Fridge"},
2 {"id":"mqtt_temp","type":"mqtt in","topic":"frigo/temp/current","qos":"2","broker":"brk1","wires":[["temp_conv","
   chart_temp","gauge_temp"]]},
3 {"id":"temp_conv","type":"function","name":"Convertir température","func":"msg.payload=Number(msg.payload);return msg;
   ","wires":[["chart_temp"]]},
4 {"id":"chart_temp","type":"ui_chart","group":"grp_temp","label":"Historique","chartType":"line","removeOlder":"3600"},
5 {"id":"gauge_temp","type":"ui_gauge","group":"grp_temp","title":"Température","min":0,"max":30},
6 {"id":"mode_select","type":"ui_dropdown","label":"Mode Médical","group":"grp_control","options":[
7   {"label":"Sang (+4 C)","value":"SANG"},{"label":"Plaquettes (+22 C)","value":"PLAQUETTES"},
8   {"label":"Vaccins (+5 C)","value":"VACCINS"},{"label":"Reactifs (10 C)","value":"REACTIFS ENZYMATIQUES"}
9 ],"wires":[["mqtt_out_mode"]]},
10 {"id":"mqtt_out_mode","type":"mqtt out","topic":"frigo/mode/set"},
11 {"id":"mqtt_door","type":"mqtt in","topic":"frigo/door/status","broker":"brk1","wires":[["door_disp","door_logic","
   influx_door"]]},
12 {"id":"door_disp","type":"ui_text","label":"État Porte","format":"{{msg.payload}}"},
13 {"id":"door_logic","type":"trigger","name":"Alerte 30s","duration":"30","units":"s","reset":"CLOSED","topic":"frigo/
   buzzer/cmd"},

```

```
14 {"id": "btn_open", "type": "ui_button", "label": "OUVRIR", "payload": "OPEN", "topic": "frigo/servo/cmd", "wires": [{"mqtt_out_cmd"},
15 {"id": "btn_close", "type": "ui_button", "label": "FERMER", "payload": "CLOSE", "topic": "frigo/servo/cmd", "wires": [{"mqtt_out_cmd"}]},
16 {"id": "mqtt_obstacle", "type": "mqtt_in", "topic": "frigo/obstacle", "qos": "2", "wires": [{"obstacle_disp", "influx_stock"}]},
17 {"id": "obstacle_disp", "type": "ui_text", "label": "Contenu", "format": "{{msg.payload}}"},
18 {"id": "mqtt_peltier", "type": "mqtt_in", "topic": "frigo/peltier/state", "wires": [{"peltier_disp", "influx_peltier"}]},
19 {"id": "mqtt_loc", "type": "mqtt_in", "topic": "frigo/location", "wires": [{"ui_position"}]},
20 {"id": "ui_position", "type": "ui_template", "format": "<div> {{msg.payload}}</div>"},
21 {"id": "d640cf22b1c0c33f", "type": "influxdb_out", "name": "Medical Fridge", "bucket": "Fridge_Data", "org": "MedicalLab"},
22 {"id": "brk1", "type": "mqtt-broker", "name": "LocalBroker", "broker": "localhost", "port": "1883"}
```